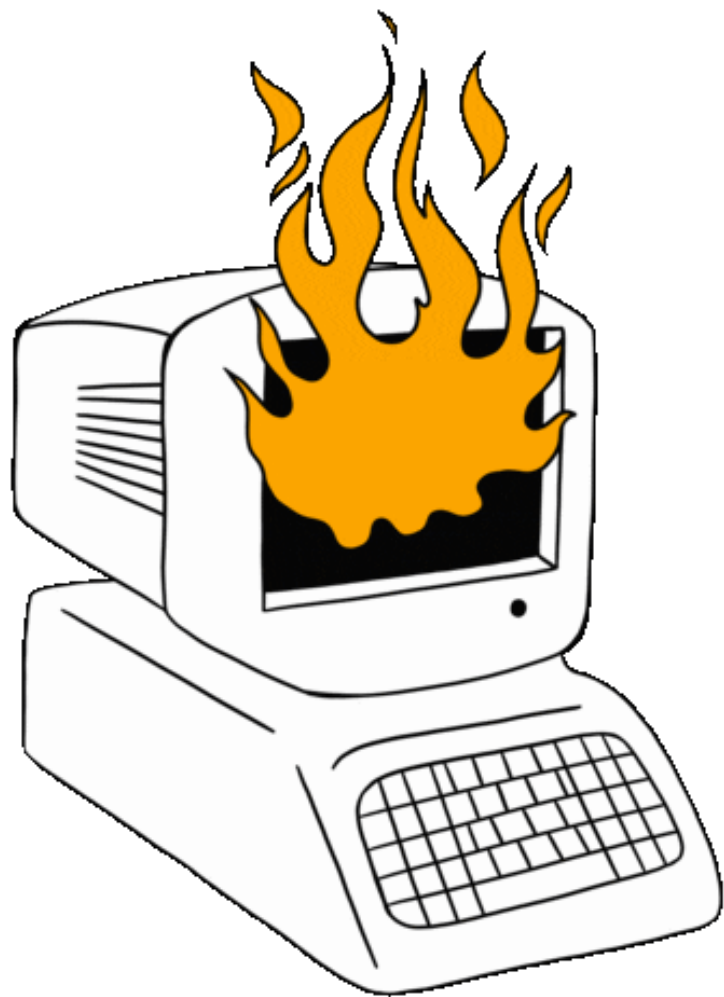


Mini SNMP Agent



Agente SNMP para monitorización de CPU.

Presentado por Marcos Fraile y Arturo Francés

Proceso de desarrollo asistido por IA

1. **Elaboración hoja de ruta.**
2. **Diagrama UML.**
3. **Desarrollo MIB.**
4. **Formato JSON.**
5. **Programa mini_agent.py**
6. **Programa test_agent.py**
7. **README.md**

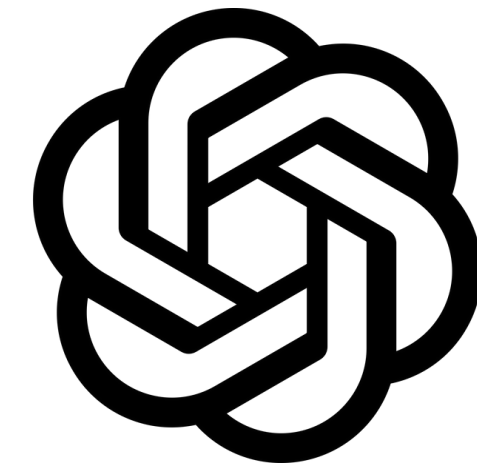
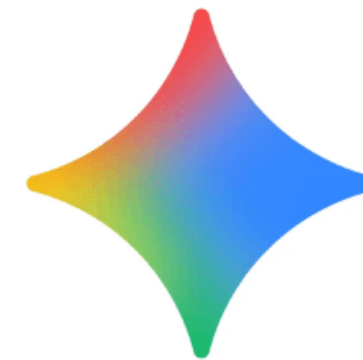
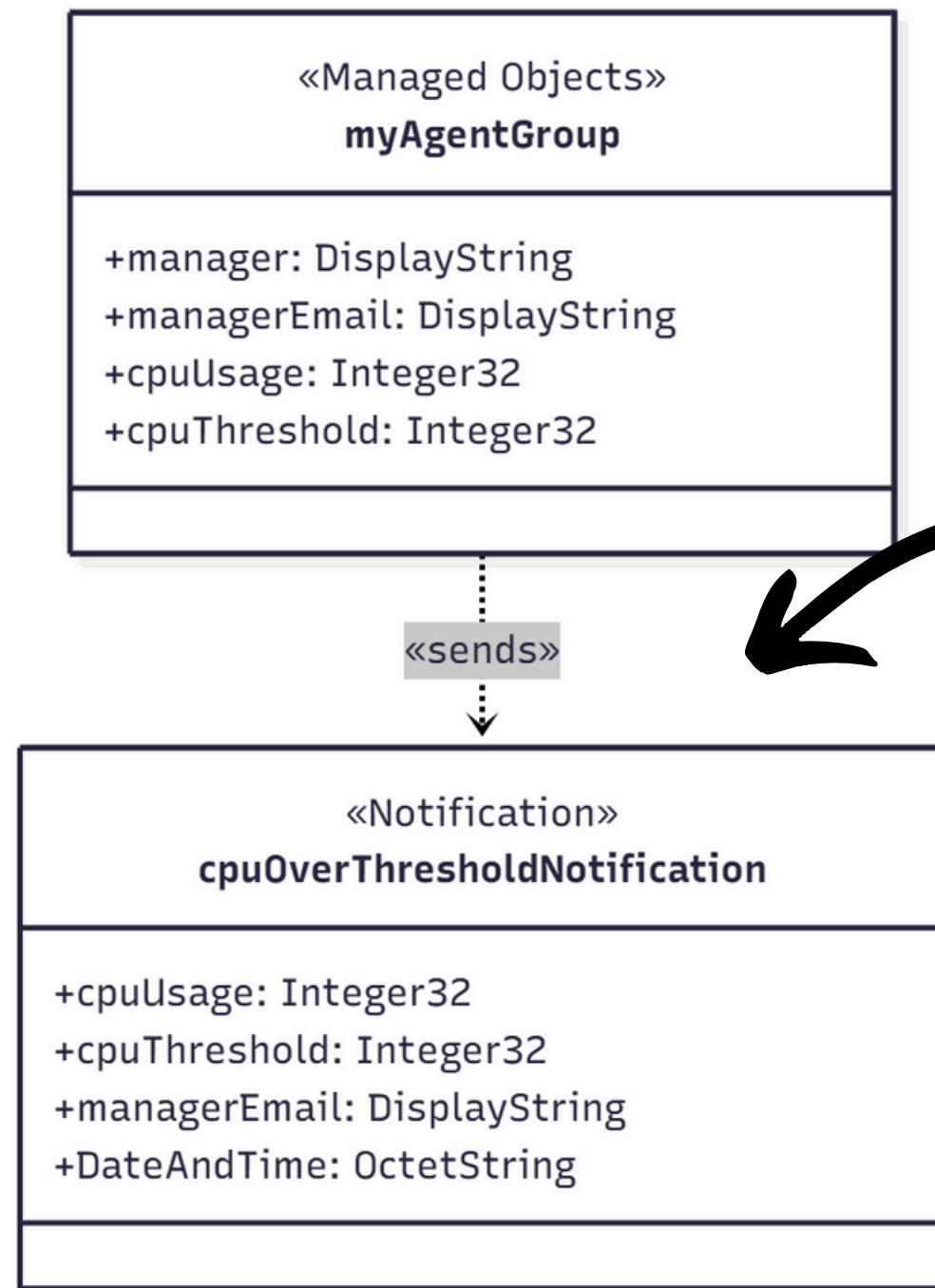


Diagrama UML

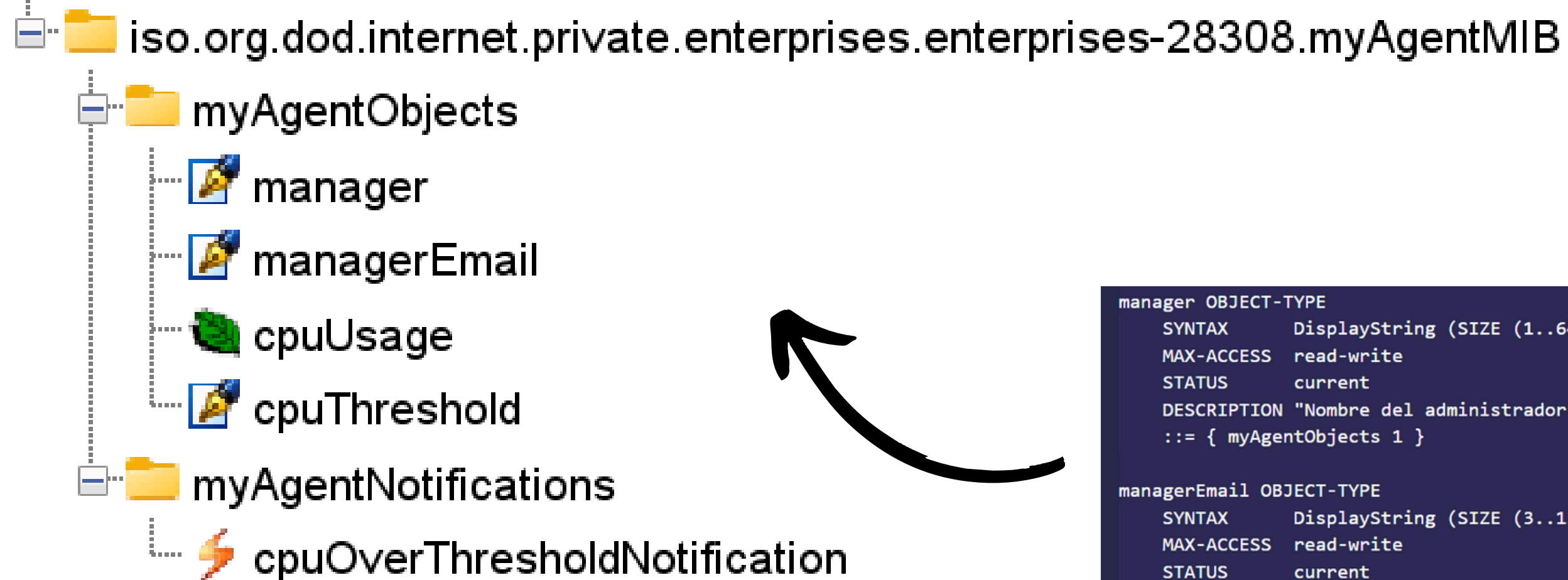


```
1 classDiagram
2   class MyAgentGroup {
3       +DisplayString manager
4       +DisplayString managerEmail
5       +Integer32 cpuUsage
6       +Integer32 cpuThreshold
7   }
8
9   class cpuOverThresholdNotification {
10      <<Notification>>
11      +cpuUsage
12      +cpuThreshold
13      +managerEmail
14      +dateAndTime
15  }
16
17  MyAgentGroup ..> cpuOverThresholdNotification : "sends"
```

MIB



MIB Tree



```
manager OBJECT-TYPE
    SYNTAX      DisplayString (SIZE (1..64))
    MAX-ACCESS  read-write
    STATUS      current
    DESCRIPTION "Nombre del administrador de la red."
    ::= { myAgentObjects 1 }

managerEmail OBJECT-TYPE
    SYNTAX      DisplayString (SIZE (3..128))
    MAX-ACCESS  read-write
    STATUS      current
    DESCRIPTION "Email del administrador para recibir las notificaciones."
    ::= { myAgentObjects 2 }

cpuUsage OBJECT-TYPE
    SYNTAX      Gauge32 (0..100) -- Using Gauge32 for non-negative, up-or-down value
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION "Uso actual de la CPU (%)."
    ::= { myAgentObjects 3 }
```

Base de datos JSON

```
{
  "base_oid": "1.3.6.1.4.1.28308.1.1",
  "scalars": {
    "manager": {
      "oid": "1.3.6.1.4.1.28308.1.1.1.0",
      "type": "DisplayString",
      "access": "read-write",
      "min_len": 1,
      "max_len": 64,
      "value": "Admin"
    },
    "managerEmail": {
      "oid": "1.3.6.1.4.1.28308.1.1.2.0",
      "type": "DisplayString",
      "access": "read-write",
      "min_len": 3,
      "max_len": 128,
      "value": "sexyAdmin@gmail.com"
    }
  },
}
```

```
    "cpuUsage": {
      "oid": "1.3.6.1.4.1.28308.1.1.3.0",
      "type": "Gauge32",
      "access": "read-only",
      "value": 4
    },
    "cpuThreshold": {
      "oid": "1.3.6.1.4.1.28308.1.1.4.0",
      "type": "Integer32",
      "access": "read-write",
      "min_val": 1,
      "max_val": 100,
      "value": 75
    }
  }
}
```


mini_agent.py

PASO 1: Gestionar almacenamiento en JSON.

```
# Inicialización y carga del archivo JSON.
def __init__(self, filename):
    self.filename = filename
    try:
        with open(filename) as f:
            self.model = json.load(f)
            print(f"Cargado estado existente desde {filename}")

    except FileNotFoundError:
        print(f"ADVERTENCIA: {filename} no encontrado. Creando uno nuevo.")
        self.model = { ... }
        self.save()

# Construimos un mapa de OIDs.
self.oid_map = {tuple(map(int, v["oid"].split("."))): k
                 for k, v in self.model["scalars"].items()}
self.sorted_oids = sorted(self.oid_map.keys())

# Guardar OIDs para el trap.
base_oid_mib = "1.3.6.1.4.1.28308.1"

self.oid_cpuUsage = tuple(map(int, self.model["scalars"]["cpuUsage"]["oid"].split(".")))
self.oid_cpuThreshold = tuple(map(int, self.model["scalars"]["cpuThreshold"]["oid"].split(".")))
self.oid_managerEmail = tuple(map(int, self.model["scalars"]["managerEmail"]["oid"].split(".")))

self.oid_notification = tuple(map(int, f"{base_oid_mib}.2.1".split(".")))
self.oid_snmpTrapOID = (1, 3, 6, 1, 6, 3, 1, 1, 4, 1, 0)
self.oid_eventTime = tuple(map(int, f"{base_oid_mib}.1.1.5.0".split(".")))
```

```
# Guardar el estado actual en el archivo JSON.
def save(self):
    with open(self.filename, "w") as f:
        json.dump(self.model, f, indent=2)
```


PASO 2: Operaciones SNMP.

Devuelve el valor asociado a un OID dado.

```
def get_exact(self, oid_tuple):
    if oid_tuple in self.oid_map:
        key = self.oid_map[oid_tuple]
        val = self.model["scalars"][key]["value"]
        typ = self.model["scalars"][key]["type"]
        return True, self._to_snmp_type(val, typ)
    return False, None
```

Devuelve el OID siguiente a uno dado y su valor.

```
def get_next(self, oid_tuple):
    for next_oid in self.sorted_oids:
        if next_oid > oid_tuple:
            key = self.oid_map[next_oid]
            val = self.model["scalars"][key]["value"]
            typ = self.model["scalars"][key]["type"]
            return True, next_oid, self._to_snmp_type(val, typ)
    return False, None, None
```

Verificamos si la operación del SET se puede ejecutar o no.

```
def validate_set(self, oid_tuple, val):

    # Error: No podemos manejar este OID.
    if oid_tuple not in self.oid_map:
        return 6, 0
    key = self.oid_map[oid_tuple]
    meta = self.model["scalars"][key]

    # Error: Es de Read-Only.
    if meta["access"] == "read-only":
        return 17, 0

    # Error: El tipo no coincide.
    if meta["type"] == "DisplayString" and not isinstance(val, v2c.OctetString):
        return 7, 0
    if meta["type"] == "Integer32" and not isinstance(val, v2c.Integer):
        return 7, 0

    # Error: El valor no está en el rango permitido.
    if meta["type"] == "DisplayString":
        s = str(val.prettyPrint())
        if not (meta.get("min_len", 0) <= len(s) <= meta.get("max_len", 255)):
            return 10, 0
    if meta["type"] == "Integer32":
        i = int(val)
        if not (meta.get("min_val", -2147483648) <= i <= meta.get("max_val", 2147483647)):
            return 10, 0

    return 0, 0
```

PASO 2: Operaciones SNMP.

```
# Función que formaliza el SET (simplemente lo guarda en el JSON).
def commit_set(self, oid_tuple, val):
    key = self.oid_map[oid_tuple]
    meta = self.model["scalars"][key]
    if meta["type"] == "DisplayString":
        meta["value"] = str(val.prettyPrint())
    else:
        meta["value"] = int(val)
    self.save()

# SET interno que actualiza el valor de la CPU.
def set_cpu_usage_internal(self, value):
    self.model["scalars"]["cpuUsage"]["value"] = int(value)

# Convierte en valor de Python a tipo SNMP.
def _to_snmp_type(self, value, typ):
    return v2c.OctetString(value.encode()) if typ == "DisplayString" else v2c.Integer(int(value))
```


PASO 3: Responders SNMP.

```
# Responder encargado de hacer Los GET.
class JsonGet(cmdrsp.GetCommandResponder):
    def handleMgmtOperation(self, snmpEngine, stateReference, contextName, PDU):
        req = v2c.apiPDU.getVarBinds(PDU)
        rsp = []
        for oid, _ in req:
            found, val = STORE.get_exact(tuple(oid))
            rsp.append((oid, val if found else v2c.NoSuchObject()))
        rspPDU = v2c.apiPDU.getResponse(PDU)
        v2c.apiPDU.setErrorStatus(rspPDU, 0)
        v2c.apiPDU.setErrorIndex(rspPDU, 0)
        v2c.apiPDU.setVarBinds(rspPDU, rsp)
        self.sendPdu(snmpEngine, stateReference, rspPDU)

# Responder encargado de hacer Los GETNEXT.
class JsonGetNext(cmdrsp.NextCommandResponder):
    def handleMgmtOperation(self, snmpEngine, stateReference, contextName, PDU):
        req = v2c.apiPDU.getVarBinds(PDU)
        rsp = []
        for oid, _ in req:
            ok, next_oid, val = STORE.get_next(tuple(oid))
            if ok:
                rsp.append((v2c.ObjectIdentifier(next_oid), val))
            else:
                rsp.append((oid, v2c.EndOfMibView()))
        rspPDU = v2c.apiPDU.getResponse(PDU)
        v2c.apiPDU.setErrorStatus(rspPDU, 0)
        v2c.apiPDU.setErrorIndex(rspPDU, 0)
        v2c.apiPDU.setVarBinds(rspPDU, rsp)
        self.sendPdu(snmpEngine, stateReference, rspPDU)
```

```
# Responder encargado de hacer Los SET.
class JsonSet(cmdrsp.SetCommandResponder):
    def handleMgmtOperation(self, snmpEngine, stateReference, contextName, PDU):
        req = v2c.apiPDU.getVarBinds(PDU)

        # Fase 1: Validamos el SET.
        for idx, (oid, val) in enumerate(req, start=1):
            err, _ = STORE.validate_set(tuple(oid), val)
            if err != 0:
                rspPDU = v2c.apiPDU.getResponse(PDU)
                v2c.apiPDU.setErrorStatus(rspPDU, err)
                v2c.apiPDU.setErrorIndex(rspPDU, idx)
                v2c.apiPDU.setVarBinds(rspPDU, req)
                self.sendPdu(snmpEngine, stateReference, rspPDU)
                return

        # Fase 2: Formalizamos el SET.
        for oid, val in req:
            STORE.commit_set(tuple(oid), val)

        # Recibimos respuesta.
        rsp = []
        for oid, _ in req:
            found, val = STORE.get_exact(tuple(oid))
            rsp.append((oid, val if found else v2c.NoSuchObject()))
        rspPDU = v2c.apiPDU.getResponse(PDU)
        v2c.apiPDU.setErrorStatus(rspPDU, 0)
        v2c.apiPDU.setErrorIndex(rspPDU, 0)
        v2c.apiPDU.setVarBinds(rspPDU, rsp)
        self.sendPdu(snmpEngine, stateReference, rspPDU)
```

PASO 4: Monitorización CPU y notificaciones.

```
# Función asíncrona que monitorea el uso de CPU y envía alertas.
async def cpu_monitor(snmpEngine):
    psutil.cpu_percent(interval=None)
    last_over = False

    ntfOrg = ntforG.NotificationOriginator()

    print("[MONITOR] El monitor de CPU está activo.")

    while True:
        await asyncio.sleep(5)

        loop = asyncio.get_running_loop()

        cpu = round(psutil.cpu_percent(interval=None))
        cpu = max(0, min(100, cpu))
        STORE.set_cpu_usage_internal(cpu)

        thr = int(STORE.model["scalars"]["cpuThreshold"]["value"])
        email = str(STORE.model["scalars"]["managerEmail"]["value"])
        over = cpu > thr
```

```
if over and not last_over:
    now_str = time.strftime("%Y-%m-%d, %H:%M:%S")
    print(f"[TRAP] CPU {cpu}% > {thr}% - Generando notificación...")

    # Construimos los VarBinds.
    varBinds = [
        (STORE.oid_snmpTrapOID, v2c.ObjectIdentifier(STORE.oid_notification)),
        (STORE.oid_cpuUsage, v2c.Integer(cpu)),
        (STORE.oid_cpuThreshold, v2c.Integer(thr)),
        (STORE.oid_managerEmail, v2c.OctetString(email)),
        (STORE.oid_eventTime, v2c.OctetString(now_str.encode('utf-8')))
    ]

    # Enviamos el TRAP.
    try:
        await loop.run_in_executor(None, ntfOrg.sendVarBinds, snmpEngine, 'my-area', None, '', varBinds)
        print(f"[TRAP] Notificación SNMP enviada a 127.0.0.1:162.")
    except Exception as e:
        print(f"[ERROR] Fallo al enviar TRAP en el executor: {e}")

    # Enviamos el EMAIL.
    try:
        await loop.run_in_executor(None, send_email, email, cpu, thr)
    except Exception as e:
        print(f"[ERROR] Fallo al enviar EMAIL en el executor: {e}")

elif not over and last_over:
    print(f"[INFO] CPU de vuelta a la normalidad: {cpu}% <= {thr}%")

last_over = over
```

PASO 5: Inicialización.

```
# Iniciamos el motor SNMP.
snmpEngine = engine.SnmpEngine()
config.addTransport(
    snmpEngine,
    udp.DOMAIN_NAME + (1,),
    udp.UdpTransport().openServerMode(("127.0.0.1", 161))
)
config.addTransport(
    snmpEngine,
    udp.DOMAIN_NAME,
    udp.UdpTransport()
)

# Indicamos las comunidades.
config.addV1System(snmpEngine, "public-area", "public")
config.addV1System(snmpEngine, "private-area", "private")

# Usamos las comunidades para asignar los permisos de antes al árbol.
for secModel in (1, 2):
    config.addVacmUser(
        snmpEngine, secModel, "public-area", "noAuthNoPriv",
        readSubTree=(1, 3, 6, 1) # RO
    )
    config.addVacmUser(
        snmpEngine, secModel, "private-area", "noAuthNoPriv",
        readSubTree=(1, 3, 6, 1),
        writeSubTree=(1, 3, 6, 1) # RW
    )

# Definimos parámetros de seguridad (qué comunidad usar).
config.addTargetParams(snmpEngine, "my-creds", "public-area", "noAuthNoPriv", 1) # 1=v1
```

```
# Definimos la dirección de destino.
config.addTargetAddr(
    snmpEngine,
    "my-area",
    udp.DOMAIN_NAME,
    ("127.0.0.1", 162),
    "my-creds",
    tagList="trap"
)

# Mapeamos el 'notificationHandle' a un tag de transporte
config.addNotificationTarget(
    snmpEngine,
    'my-area',
    'trap',
    'trap'
)

# Conectamos los responders JSON al motor SNMP.
snmpContext = context.SnmpContext(snmpEngine)
JsonGet(snmpEngine, snmpContext)
JsonGetNext(snmpEngine, snmpContext)
JsonSet(snmpEngine, snmpContext)

# Lo iniciamos.
loop = asyncio.get_event_loop()
loop.create_task(cpu_monitor(snmpEngine))
print("Mini SNMP Agent (versión corregida) corriendo en udp:161 (Ctrl+C to stop)")
try:
    loop.run_forever()
except KeyboardInterrupt:
    print("Agent stopped.")
```

test_agent.py

```
# Función que realiza un GET.
async def snmp_get(oid, community=COMMUNITY_RO):
    target = await UdpTransportTarget.create(AGENT, timeout=1, retries=2)
    errorIndication, errorStatus, errorIndex, varBinds = await getCmd(
        SnmpEngine(),
        CommunityData(community, mpModel=1),
        target,
        ContextData(),
        ObjectType(ObjectIdentity(oid))
    )

    if errorIndication:
        print(f"[GET] Error: {errorIndication}")
        return None
    elif errorStatus:
        print(f"[GET] {errorStatus.prettyPrint()} at {errorIndex}")
        return None
    else:
        for oid, val in varBinds:
            print(f"[GET] {oid.prettyPrint()} = {val.prettyPrint()}")
            return val.prettyPrint()
```

```
# Función que realiza un SET.
async def snmp_set(oid, value, type_tag, community=COMMUNITY_RW):
    target = await UdpTransportTarget.create(AGENT, timeout=1, retries=2)
    errorIndication, errorStatus, errorIndex, varBinds = await setCmd(
        SnmpEngine(),
        CommunityData(community, mpModel=1),
        target,
        ContextData(),
        ObjectType(ObjectIdentity(oid), type_tag(value))
    )

    if errorIndication:
        print(f"[SET] Error: {errorIndication}")
    elif errorStatus:
        print(f"[SET] Error: {errorStatus.prettyPrint()} at index {errorIndex}")
    else:
        for oid, val in varBinds:
            print(f"[SET] {oid.prettyPrint()} = {val.prettyPrint()}")
```

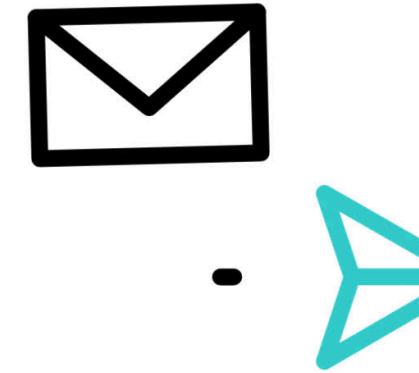

Ejecución

```
Opciones:
  1. GET
  2. GETNEXT
  3. SET
  4. WALK
  5. Volver al menú principal.

Opción: 1

Posibles valores de OID:
manager: 1.3.6.1.4.1.28308.1.1.1.0
managerEmail: 1.3.6.1.4.1.28308.1.1.2.0
cpuUsage: 1.3.6.1.4.1.28308.1.1.3.0
cpuThreshold: 1.3.6.1.4.1.28308.1.1.4.0
base_subtree: 1.3.6.1.4.1.28308.1.1

Introduce el nombre del OID a leer: manager
[GET] SNMPv2-SMI::enterprises.28308.1.1.1.0 = Marcos-Agente
```



 Entrada  

 **yankmar14@gmail.com** ayer
Para: marcosfraile2004@gmail.com >

Alerta SNMP: CPU 79% > 75%

El agente SNMP ha detectado que el uso de CPU (79%) superó el umbral (75%) a las 2025-11-15, 12:07:45.